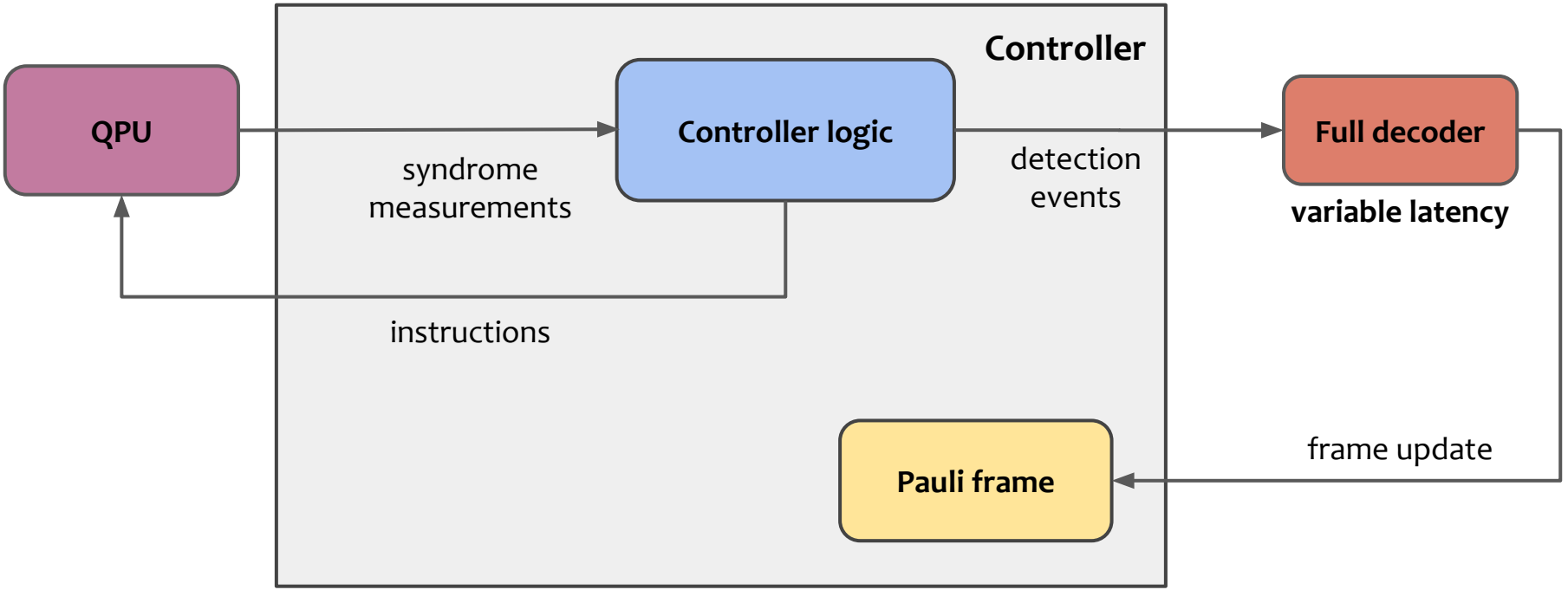
Neural-Network-Based Lightweight Predecoder for Speculative Pauli Frame Updates

Andrei Dolmatov
Advisor: Aleksandra Świerkowska
Chair of Computer Systems
https://dse.in.tum.de/

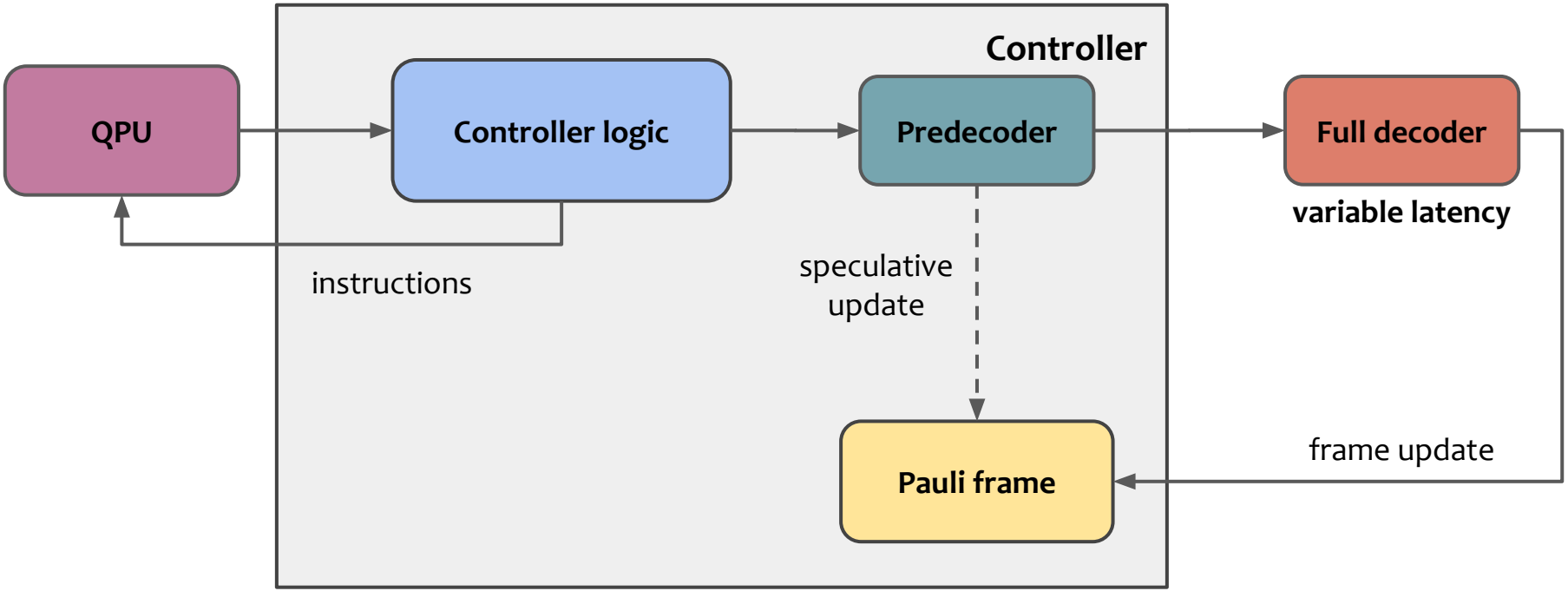


Context



Decoder delays can stall operations that depend on the decoded result

Context



Speculative NN-based updates may reduce the workload of the full decoder

Research question

Can neural predecoding reduce classical-decoder workload while preserving prediction accuracy?

Related work

- Promatch — Alavisamani et al. (2024) - Greedy predecoding simplifies surface-code syndromes before full decoding
- Pinball — Knapen et al. (2026) - Cryogenic predecoding handles simple errors locally and sends complex cases to the full decoder
- Arqade — Knapen et al. (2026, preprint) - Automatically generates predecoders for quantum LDPC codes to reduce full-decoder workload

Our focus: confidence-based neural predecoding on experimental surface-code data, evaluating accuracy and avoided decoder calls

Our proposal

A confidence-based lightweight neural predecoder with a classical decoder fallback

Key ideas

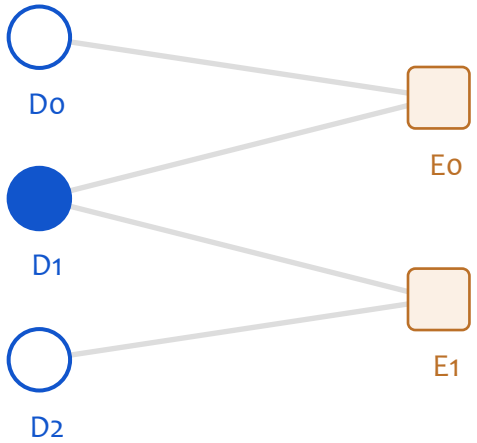
- Neural network to predict the logical flip and its confidence
- Accept sufficiently confident predictions for speculative Pauli-frame updates
- Send uncertain shots to the classical decoder

Design goals

- High accuracy
- Low-latency
- High-throughput

Detector–error graph

Detectors **Error mechanisms**



13 features per node

- Node type, position and time
- Error probability, support flags
- Observed detector event bit

- Graph structure is static
- Feature vectors change dynamically from shot to shot

Detector–error relationships naturally fit graph neural networks

Dataset and prediction task

Google Sycamore Surface-Code Dataset

- distance-five code
- X-basis
- 5-, 9-, 25-rounds
- 53 samples with 75000 shots each

Circuit + DEM

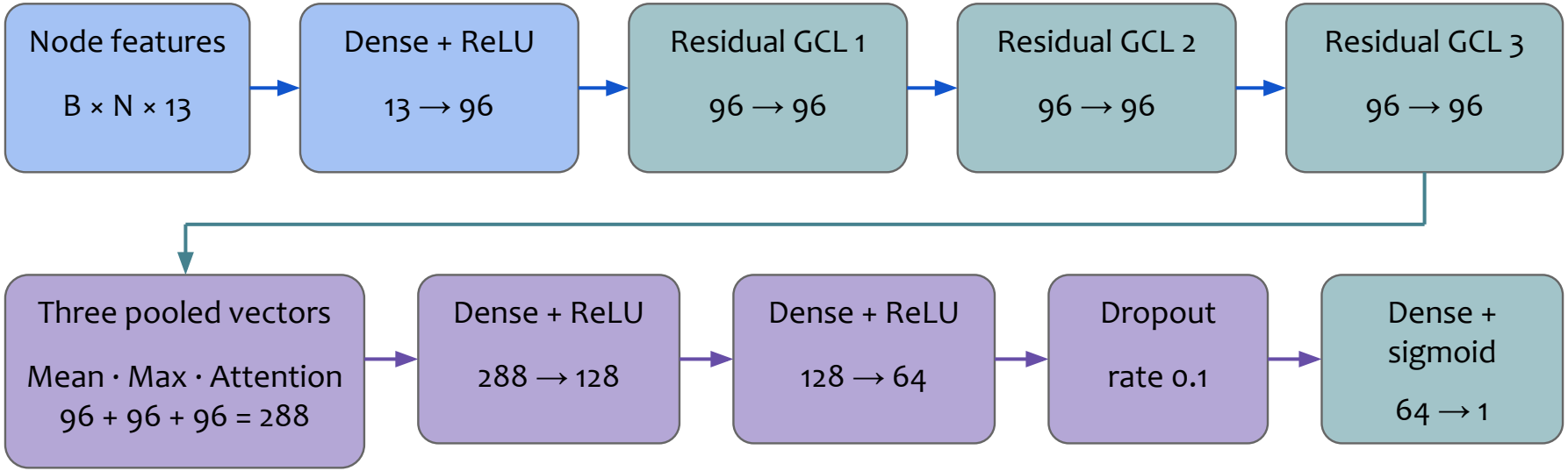
Detection events

Logical-flip label

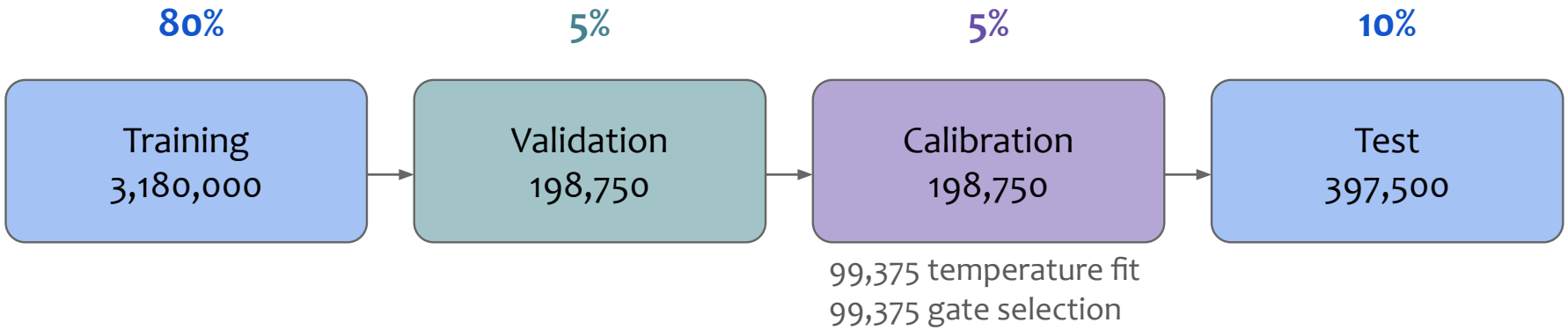
How can this data be used to predict the flip of logical observable?

Sivak et al. (2024)

Neural predecoder architecture



Training and evaluation



Fixed data mode

- same samples each epoch
- 120k unique shots
- up to 100 epochs

Fresh data mode

- same samples each epoch
- 120k unique shots
- up to 27 epochs

Model compression

Key idea: preserve the graph model, reduce its channel widths

Model	Approach	Dense head	Parameters
Baseline	-	128 -> 64 -> 1	75,266
Pruned	Remove small-weight channels	64 -> 32 -> 1	28,290 (62.4% fewer)
Pruned, Mixed precision	FP32 -> FP16	64 -> 32 -> 1	28,290 (80.8% smaller size)

Implementation



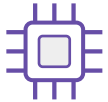
Stim + NumPy

Circuits, DEMs, packed records



Graph cache

Static features + indexed edges



TensorFlow / Keras

Message passing + prediction

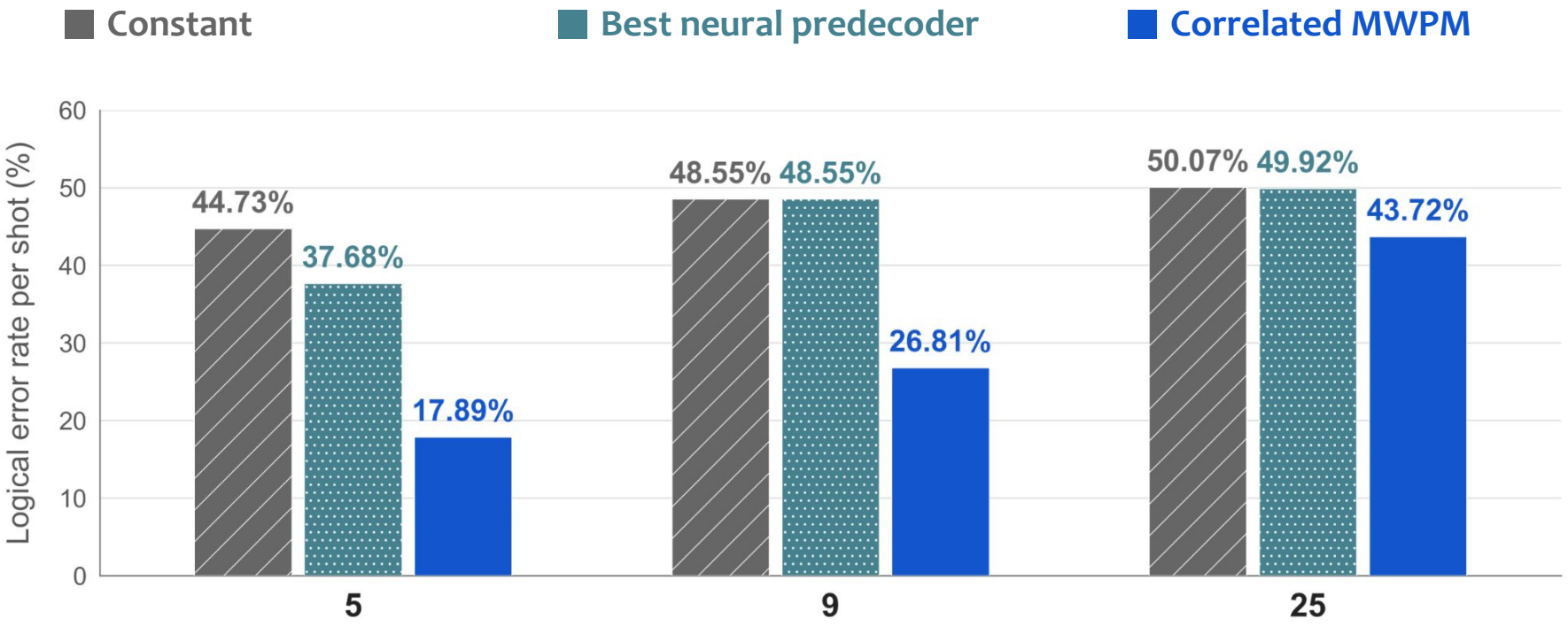
Evaluation setup

2 schedules, 2 batch sizes, 3 variants
12 variants per round count, 36 evaluated variants overal

Measurement	Configuration
Prediction	397,500 matched test shots per round
Execution	NVIDIA H200 NVL - TensorFlow 2.20 - Docker
Latency	Batch 1 - 50 warm-ups - 500 calls / sample
Classical reference	Stored correlated-matching predictions

Logical error rate per shot

lower - better



Original confidence gate: zero accepted shots

0%

classical decoder calls avoided

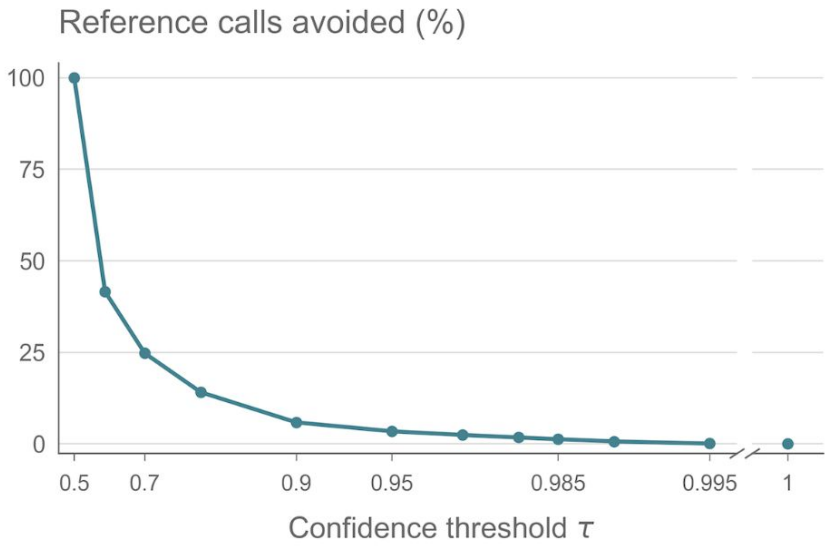
Calibration example

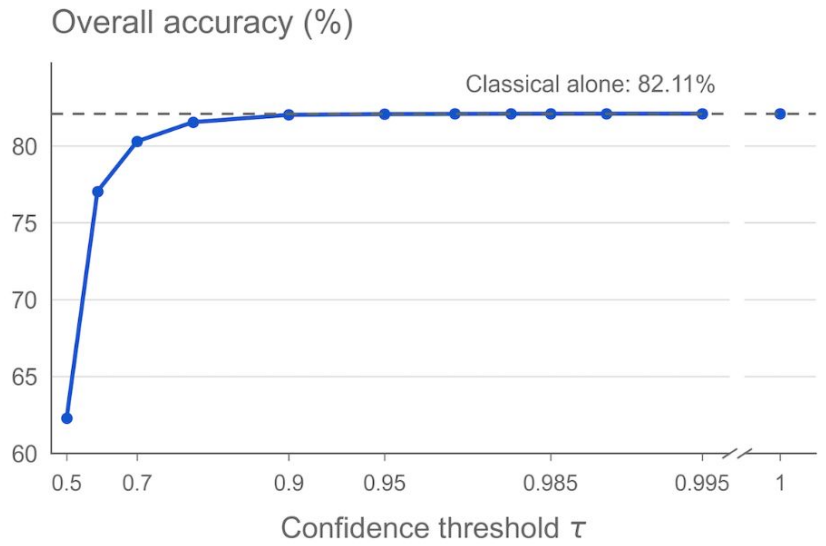
1,268 accepted shots · 21 errors
Observed error: 1.66%
Adjusted upper bound: 3.19%

All 36 fitted confidence gates delegate every shot at the 1% error target

Confidence threshold: workload and accuracy

higher - better





Near-equal overall accuracy can still hide extra errors

Empirical example: confidence threshold 0.985

5,103 accepted out of 397,500 test shots - **1.284%** reference calls avoided

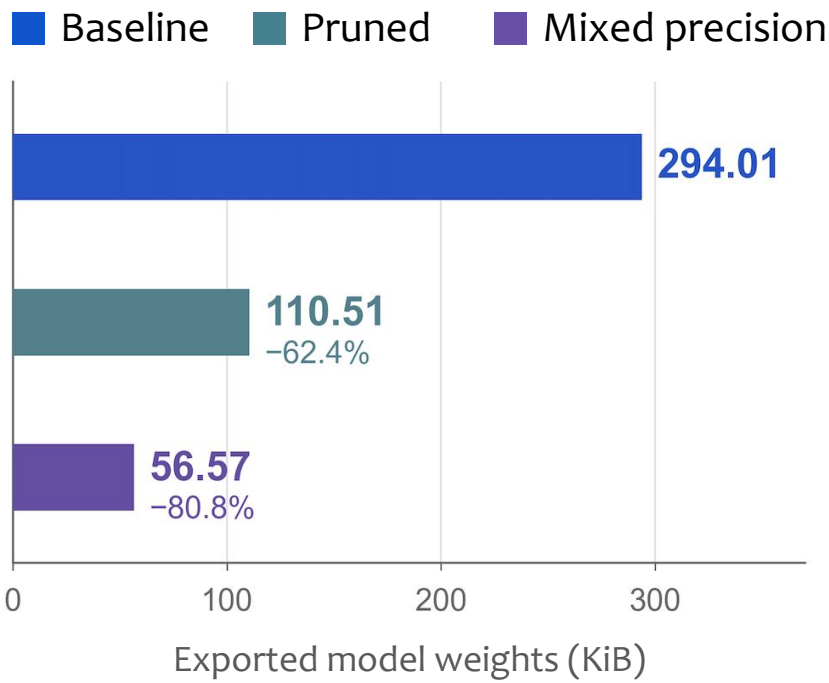
Corrected
15 errors

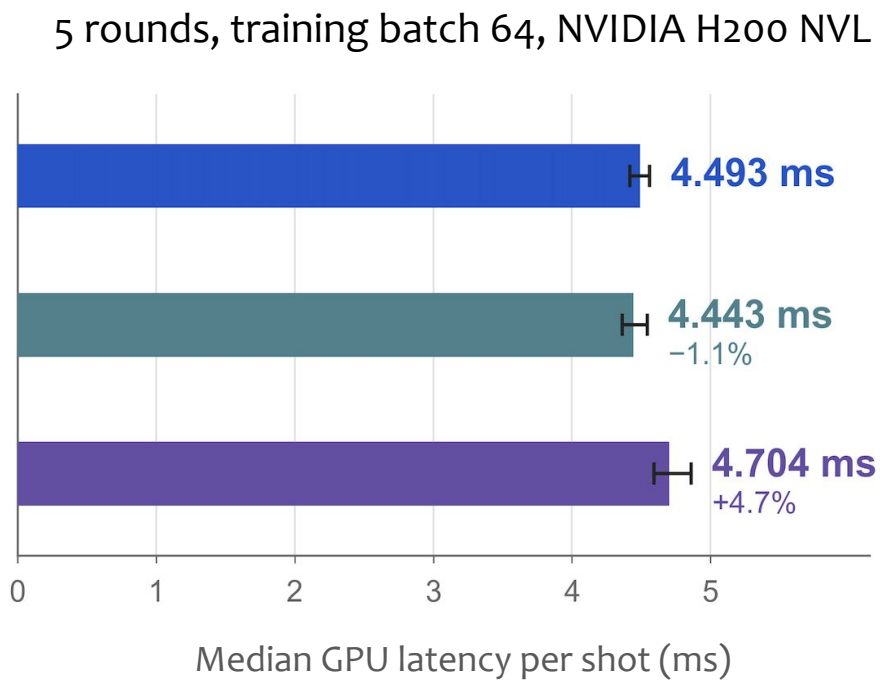
Introduced
33 errors

Accuracy on accepted subset:
Neural predecoder: 98.37%
Classical decoder: 98.73%

Reducing the confidence threshold does not necessarily lead to higher overall accuracy

Compression: storage and latency





Pruning reduces latency by 1.1%, mixed precision increases it by 4.7%
62.4% memory reduction

Limits and next experiments

Limitation	Possible solution
No reliable acceptance	Use paired neural and classical predictions for thresholds
Limited prediction accuracy	NN architectures that can capture longer syndromes
No demonstrated overall speedup	FPGA deployment
Limited generalization evidence	Evaluate additional code distances, measurement bases, samples, training seeds
Extremely slow training on full dataset	Parallelization

Conclusion

Can neural predecoding reduce classical-decoder workload while preserving prediction accuracy?

- **Best accuracy:** 62.32% (5 rounds), compared with 82.11% for classical decoding
- **GPU latency:** 4.44 ms per shot on an NVIDIA H200 NVL GPU
- **Model-weight memory:** 62.4% reduction
- **Reliable workload reduction:** all shots still require classical decoding under the 1% error target; relaxed confidence avoids 1.28% of classical calls, but adds 18 errors

Demonstrated proof of concept,
reliable acceptance and a runtime advantage remain to be established

Backup: layers and parameter counts

Stage	Shape / transform	Parameters
Input projection	13 → 96, ReLU	1,344
3 × GCN + LayerNorm	96 → 96	27,936 + 576
Mean / max / attention	96 + 96 + 96 → 288	97
Dense 1	288 → 128, ReLU	36,992
Dense 2	128 → 64, ReLU	8,256
Output	64 → 1, sigmoid	65
Total		75,266

Backup: all 5-round variants

Schedule / batch / variant	Accuracy	AUC	Latency
fresh / 64 / baseline	58.075%	0.6360	4.435 ms
fresh / 64 / pruned	59.327%	0.6501	4.391 ms
fresh / 64 / mixed	59.329%	0.6502	4.830 ms
fresh / 128 / baseline	58.584%	0.6373	4.439 ms
fresh / 128 / pruned	59.171%	0.6470	4.439 ms
fresh / 128 / mixed	59.175%	0.6470	4.811 ms
fixed / 64 / baseline	62.306%	0.6908	4.493 ms
fixed / 64 / pruned	62.316%	0.6904	4.443 ms
fixed / 64 / mixed	62.314%	0.6904	4.704 ms
fixed / 128 / baseline	60.040%	0.6614	4.495 ms
fixed / 128 / pruned	60.076%	0.6617	4.411 ms
fixed / 128 / mixed	60.076%	0.6617	4.694 ms

Backup: all 9-round variants

Schedule / batch / variant	Accuracy	AUC	Latency
fresh / 64 / baseline	51.450%	0.5096	4.519 ms
fresh / 64 / pruned	51.450%	0.4916	4.458 ms
fresh / 64 / mixed	51.450%	0.4916	4.647 ms
fresh / 128 / baseline	51.450%	0.4991	4.404 ms
fresh / 128 / pruned	51.450%	0.5070	4.438 ms
fresh / 128 / mixed	51.450%	0.5071	4.625 ms
fixed / 64 / baseline	51.450%	0.4930	4.461 ms
fixed / 64 / pruned	51.450%	0.4971	4.403 ms
fixed / 64 / mixed	51.450%	0.4972	4.713 ms
fixed / 128 / baseline	51.450%	0.5030	4.399 ms
fixed / 128 / pruned	51.450%	0.4961	4.440 ms
fixed / 128 / mixed	51.450%	0.4961	4.649 ms

Backup: thresholds and paired errors

τ	Avoided calls	Hybrid acc.	Introduced	Corrected	Δ acc. (pp)
0.5	100.000%	62.30591%	111589	32866	-19.80453
0.7	24.758%	80.30415%	10560	3380	-1.80629
0.9	5.867%	82.02642%	550	216	-0.08403
0.95	3.439%	82.07799%	213	84	-0.03245
0.97	2.446%	82.09258%	117	46	-0.01786
0.98	1.749%	82.10088%	64	26	-0.00956
0.985	1.284%	82.10591%	33	15	-0.00453
0.99	0.678%	82.10893%	12	6	-0.00151
0.995	0.108%	82.11019%	1	0	-0.00025
1	0.000%	82.11044%	0	0	+0.00000

Backup: measurement and uncertainty

- Latency: batch 1; host input; synchronous output materialization.
- Scopes: model only → detector-to-prediction → detector-to-gate.
- Throughput: batch 8, five passes; parameter payload: exported arrays.
- Paired interval: introduced / corrected counts; simultaneous bounds.
- Interpretation assumes independent shots; threshold sweep is exploratory.

Backup: selected references

Google Quantum AI (2023)

Surface-code experiments · doi:10.1038/s41586-022-05434-1

Sivak, Newman & Klimov (2024)

Decoder priors and dataset · doi:10.1103/PhysRevLett.133.150603

Torlai & Melko (2017); Lange et al. (2025)

Neural / graph decoders · PRL 119, 030501; PRResearch 7, 023181

Gidney (2021)

Stim · doi:10.22331/q-2021-07-06-497

Google Quantum AI and Collaborators (2025)

Real-time QEC · doi:10.1038/s41586-024-08449-y

Clopper & Pearson (1934); NIST/SEMATECH

Binomial bounds · doi:10.1093/biomet/26.4.404; Bonferroni method

Backup: selected references

Alavisamani et al. (2024)

Promatch: adaptive predecoding · doi:10.1145/3620666.3651339

Knapen et al. (2026)

Pinball: cryogenic predecoding · doi:10.1109/HPCA68181.2026.11408464

Knapen et al. (2026, preprint)

Arqade: generalized qLDPC predecoding · arXiv:2605.03180